# Fraudes em Cartões

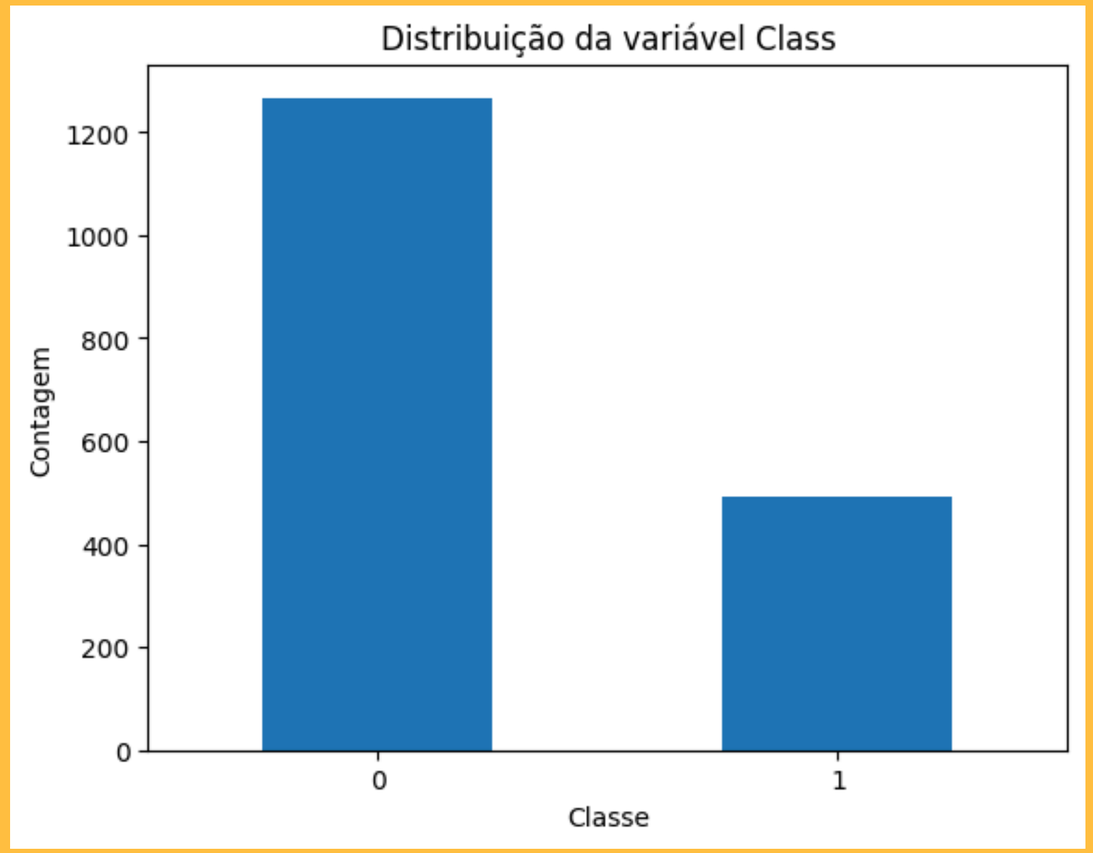
Gregory de Oliveira
João Otávio Gurski Castro

# Introdução ao DataSet

- Features - 31
- Tamanho – 1.759 registros.

# Carregando DataSet

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report, f1_score
import warnings
warnings.filterwarnings("ignore")

csv_path = "creditcard - menor balanceado.csv"
df = pd.read_csv(csv_path)
print("Arquivo carregado de:", csv_path)
print("\n--- 5 primeiras linhas ---")
display(df.head())
print("\n--- Info ---")
print(df.info())
print("\n--- Distribuição da variável 'Class' ---")
dist = df['Class'].value_counts().rename_axis('Class').reset_index(name='counts')
dist['perc'] = 100 * dist['counts'] / dist['counts'].sum()
display(dist)
```




```
--- Distribuição da variável 'Class' ---
```

| | Class | counts | perc |
|---|---|---|---|
| 0 | 0 | 1267 | 72.029562 |
| 1 | 1 | 492 | 27.970438 |

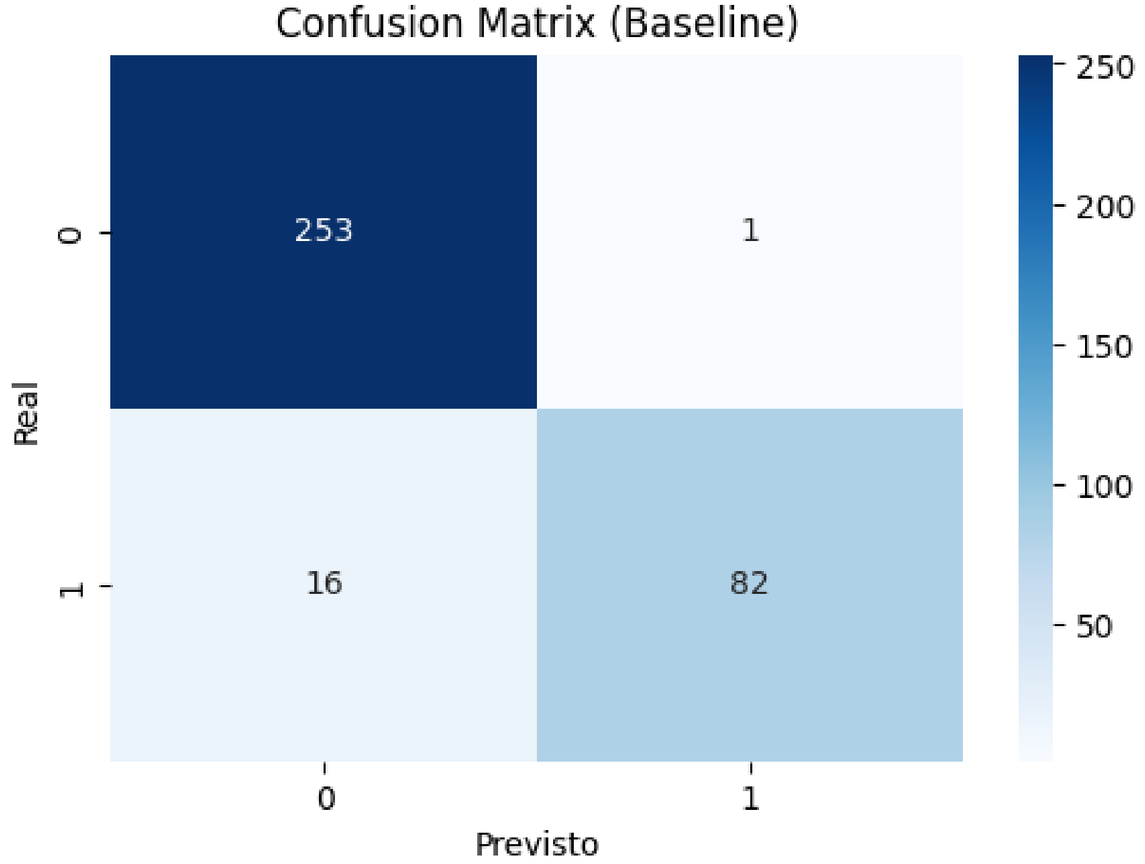
# Preparando Dados

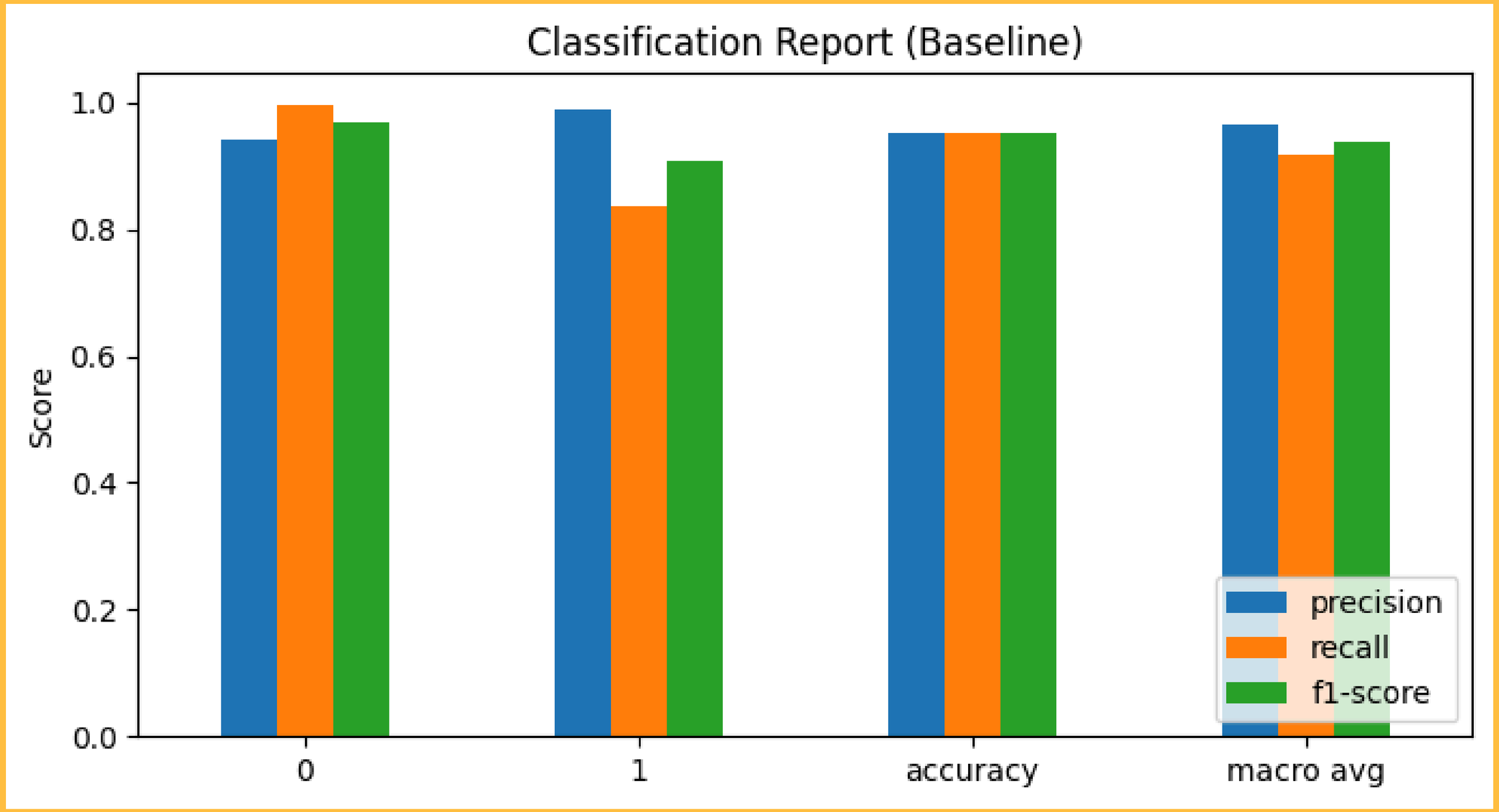
```
data = df.copy()
scaler = StandardScaler()
for col in ['Time', 'Amount']:
    if col in data.columns:
        data[col] = scaler.fit_transform(data[[col]])
X = data.drop(columns=['Class'])
y = data['Class']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
print(f"\nShapes -> X_train: {X_train.shape}, X_test: {X_test.shape}, y_train: {y_train.shape}, y_test: {y_test.shape}")
```

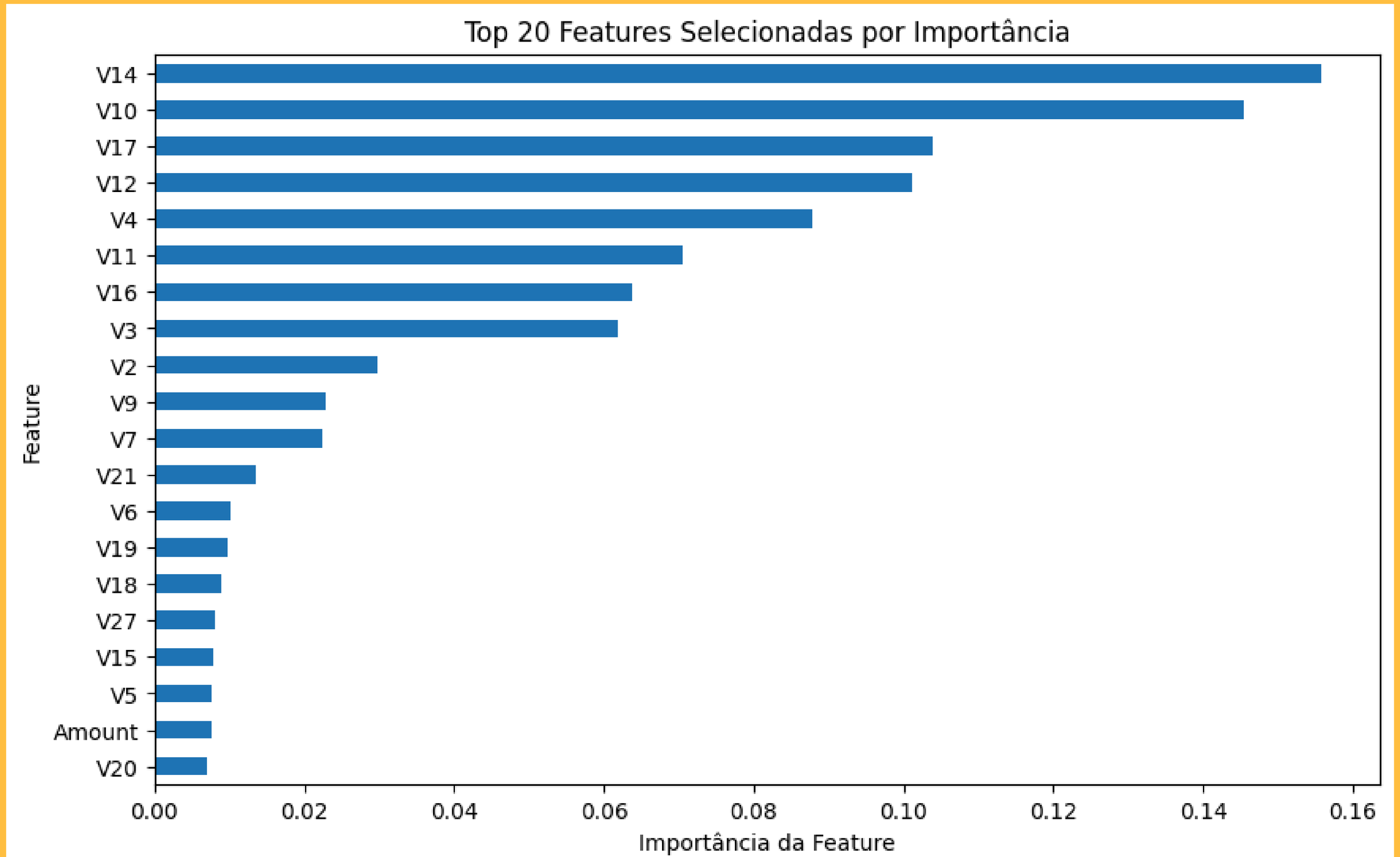
```
Shapes -> X_train: (1407, 30), X_test: (352, 30), y_train: (1407,), y_test: (352,)
```

# Modelo Base - Random Forest Classifier

```
--- Baseline: RandomForest (padrão) ---
Confusion Matrix:
[[253   1]
 [ 16  82]]

Classification Report (Baseline):
              precision    recall  f1-score   support

           0     0.9405    0.9961    0.9675       254
           1     0.9880    0.8367    0.9061        98

    accuracy                         0.9517       352
   macro avg     0.9642    0.9164    0.9368       352
weighted avg     0.9537    0.9517    0.9504       352

Macro F1 (Baseline): 0.93678628397578 78
```




Classification Report (Baseline)
Score
1.0
0.8
0.6
0.4
0.2
0.0
0
1
accuracy
macro avg
precision
recall
f1-score

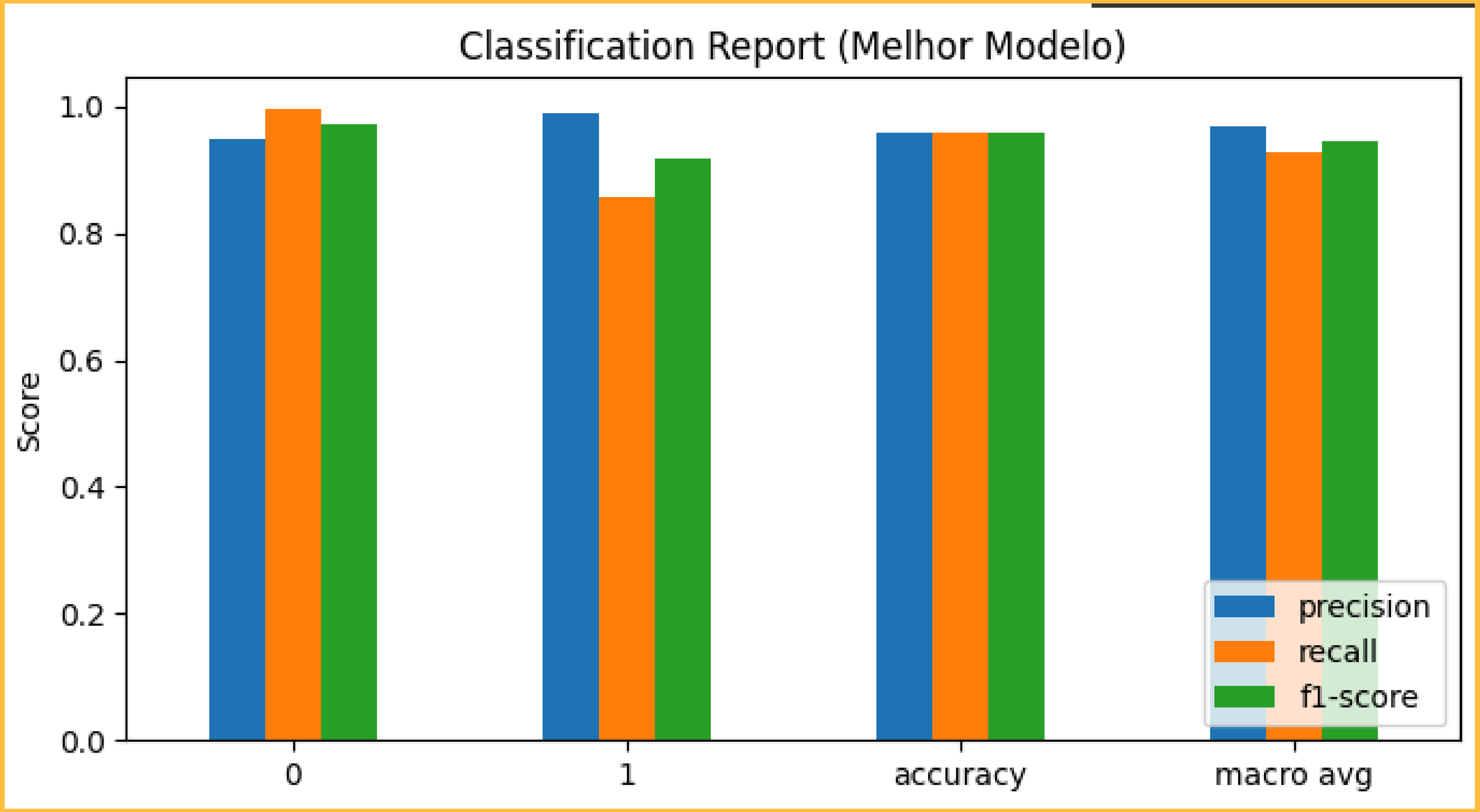

Top 20 Features Selecionadas por Importância
V14
V10
V17
V12
V4
V11
V16
V3
V2
V9
V7
V21
V6
V19
V18
V27
V15
V5
Amount
V20
Feature
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
0.16
Importância da Feature

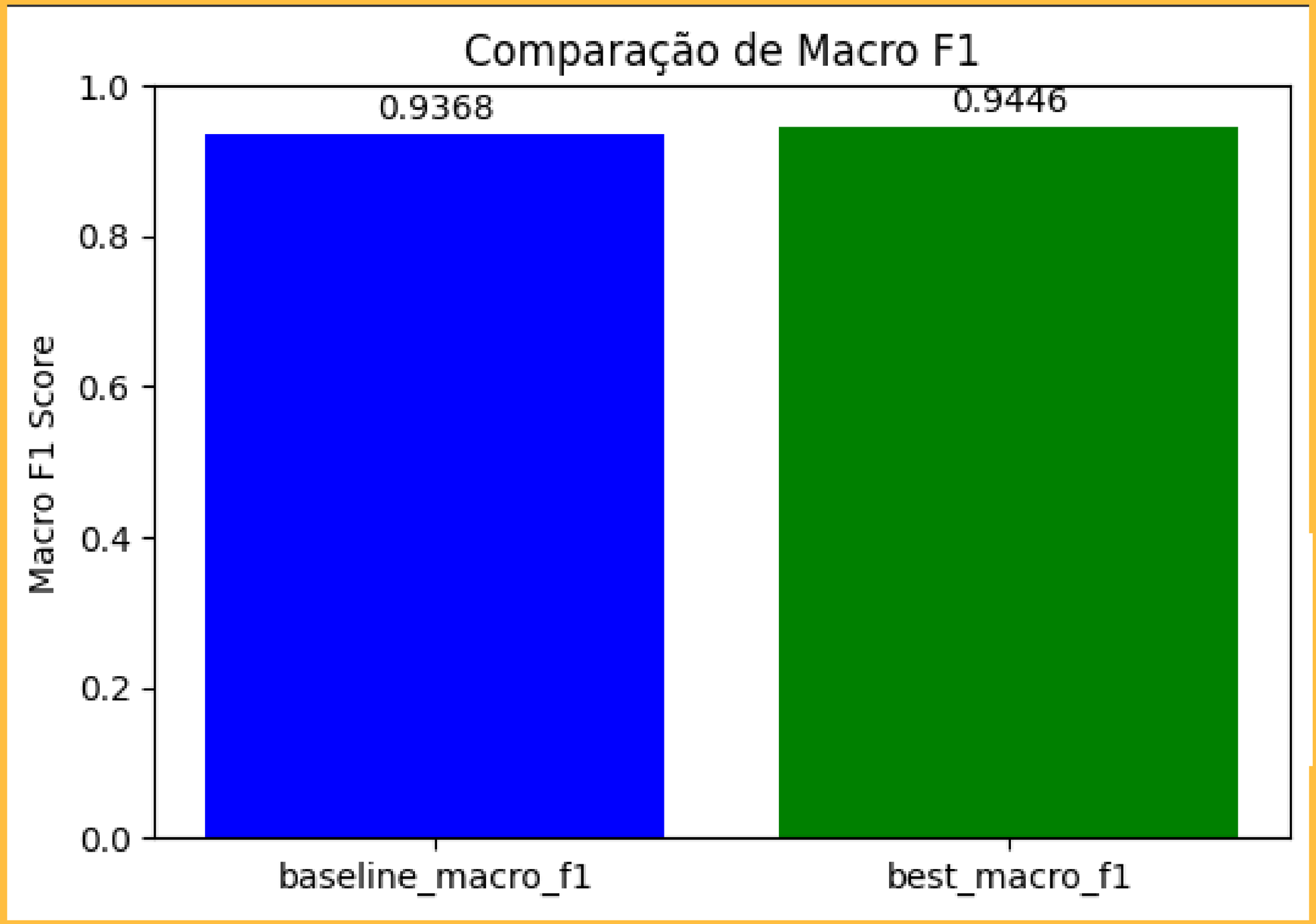
# FS + Balanceamento + Tuning

```python
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_bal, y_train_bal = smote.fit_resample(X_train_fs, y_train)
method_used = "SMOTE"

display(pd.Series(y_train_bal).value_counts())
```

| Class | count |
|---|---|
| 0 | 1013 |
| 1 | 1013 |

```python
from sklearn.model_selection import RandomizedSearchCV

print("Iniciando Tuning com RandomizedSearchCV...")

param_dist = {'n_estimators': [100, 200, 300, 500],
              'max_features': ['sqrt', 'log2'],
              'max_depth': [10, 20, 30, None],
              'min_samples_split': [2, 5, 10],
              'min_samples_leaf': [1, 2, 4],
              'bootstrap': [True, False]}

random_search = RandomizedSearchCV(RandomForestClassifier(random_state=42, n_jobs=-1),
                                   param_distributions=param_dist,
                                   n_iter=10,
                                   cv=5,
                                   scoring='f1_macro',
                                   random_state=42,
                                   n_jobs=-1,
                                   verbose=1)

random_search.fit(X_train_bal, y_train_bal)

best_clf = random_search.best_estimator_
```

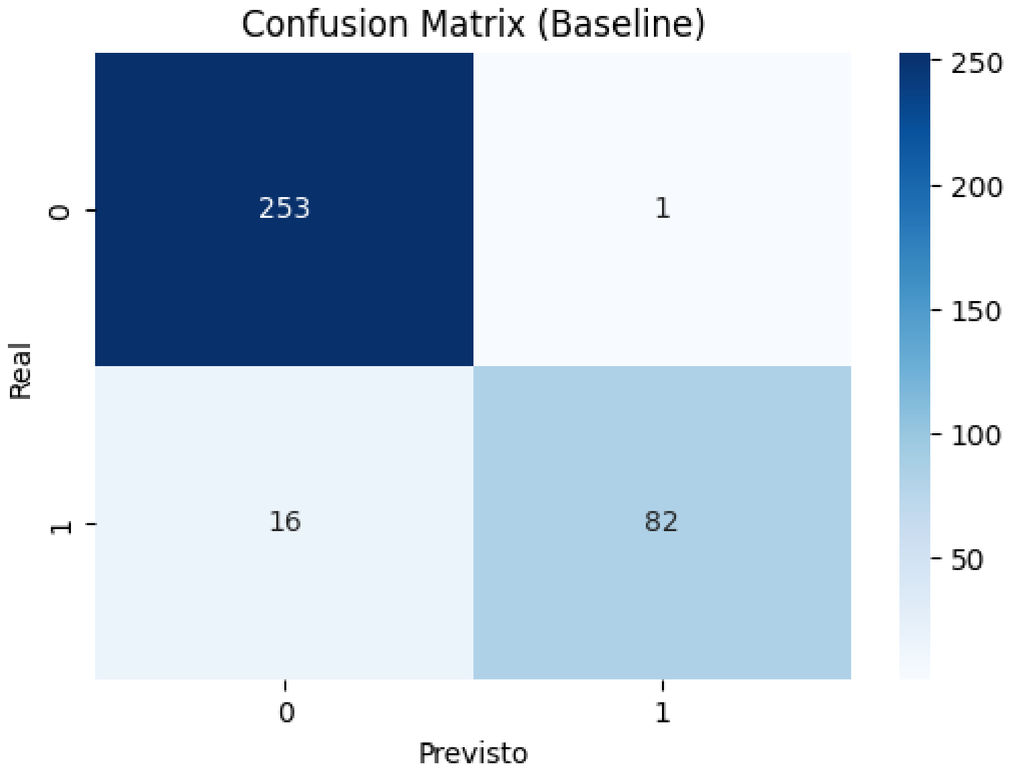

Classification Report (Melhor Modelo)
Score
1.0
0.8
0.6
0.4
0.2
0.0
0
1
accuracy
macro avg
precision
recall
f1-score

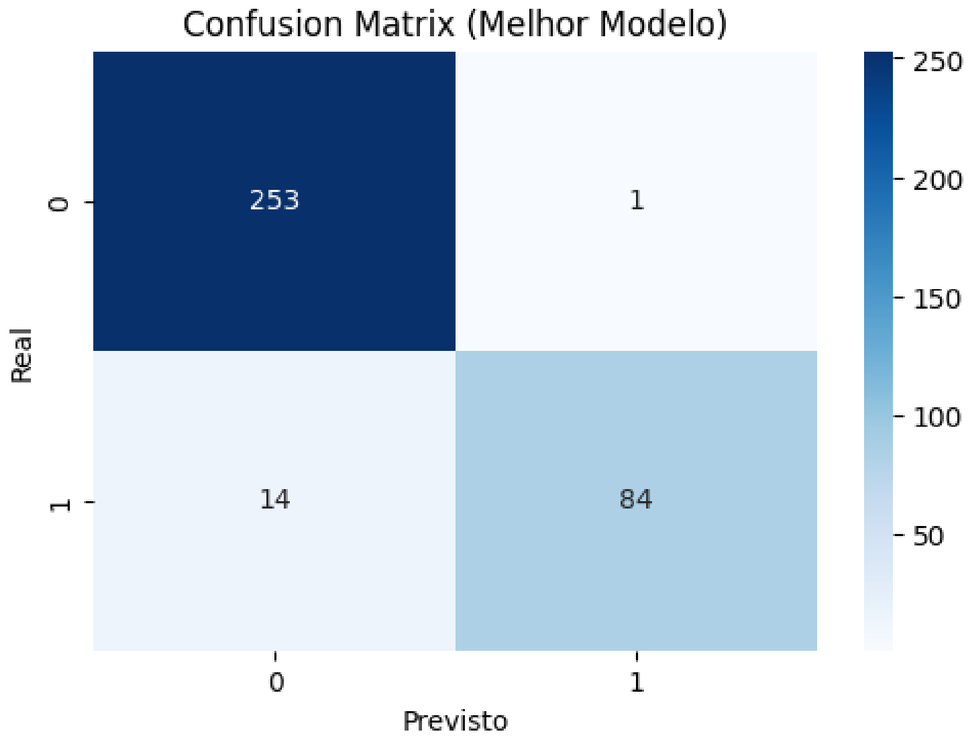

Comparação de Macro F1
Macro F1 Score
1.0
0.8
0.6
0.4
0.2
0.0
0.9368
0.9446
baseline_macro_f1
best_macro_f1

**Comparativo de Métricas:**

- Acurácia (Baseline): 0.9517
- Acurácia (Melhor Modelo Tunado): 0.9574

- Macro Avg F1-Score (Baseline): 0.9368
- Macro Avg F1-Score (Melhor Modelo Tunado): 0.9446



# Obrigado